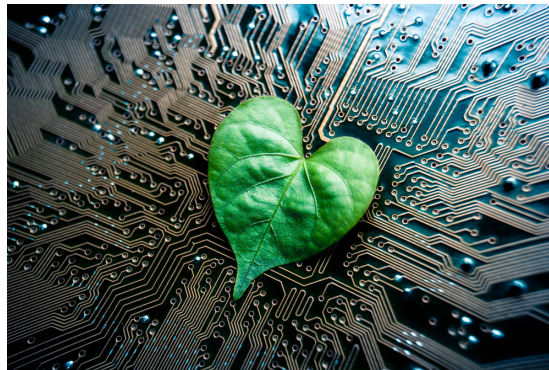




Mini-projet Numérique Durable 2026

CHAMPAIN Ayuna, DAMBRON Max, CORNUAU Nathan,
DANJOU Thomas, BACHELIER Dennis



Contents

1	Introduction - présentation du projet	4
1.1	Proposition de valeur	4
1.2	Périmètre MVP	4
1.3	Utilisateurs cibles	4
1.4	Contraintes Green IT	5
2	Architecture et conception	6
2.1	Architecture générale	6
2.2	Schéma de la base de données	6
2.3	Diagrammes UML	7
2.3.1	Diagramme de classes	7
2.3.2	Diagramme de cas d'utilisation	8
2.3.3	Diagramme de séquence — Création d'une offre	9
2.4	Architecture des pages frontend	10
3	Choix technologiques	11
3.1	Technologies choisies	11
3.2	Technologies écartées	11
4	Implémentation	12
4.1	Fonctionnalités développées	12
4.2	Extraits de code	12
4.3	Premier extrait - Requête SQL filtrée dynamique (offres.php)	12
4.4	Deuxième extrait - CRUD offres (action create, offres.php)	13
4.5	Troisième extrait - Fetch JavaScript, chargement des offres (accueil.html)	13
4.6	Quatrième extrait - Suppression en cascade utilisateur (utilisateurs.php)	14
4.7	Structure du projet	14
5	Analyse de l'empreinte carbone	15
5.1	Méthodologie de mesure	15
5.2	Résultats	15
5.2.1	Website Carbon Calculator — RecycleMarché	17
5.2.2	Comparaison avec Vinted	17
5.3	Identification des sources de pollution numérique	18
5.4	Interprétation des résultats	19
6	Tests et validation	20
6.1	Scénarios de tests	20
6.2	Résultats Lighthouse	21
6.2.1	Méthodologie	21
6.2.2	Résultats globaux	21
6.2.3	Core Web Vitals	21

6.2.4	Points d'amélioration identifiés	21
7	Organisation de l'équipe et collaboration	23
7.1	Répartition des tâches	23
7.2	Méthode de collaboration	23
8	Discussion et conclusion	24
8.1	Défis	24
8.2	Compromis	24
8.3	Améliorations	25
8.4	Critique	25
8.5	Conclusion	25
9	Annexes	26
9.1	Schéma SQL	26
9.2	Données de test	26

1 Introduction - présentation du projet

1.1 Proposition de valeur

RecycloMarché est un marché en ligne communautaire dédié à la vente et à l'achat de biens de seconde main entre particuliers. Notre proposition repose sur 2 piliers fondamentaux :

- Sobriété numérique : chargement minimal des ressources, peu voire pas d'images, aucune publicité
- Transparence : données collectées minimales (nom, prénom, adresse mail, code postal)

La plateforme s'inscrit dans une démarche Green IT, plus que nécessaire aujourd'hui. Cela permet aussi à l'utilisateur de profiter d'une expérience plus épurée (pas de publicités, pas d'images parasites, bandeau en bas de page rappelant les engagements de RecycloMarché)

1.2 Périmètre MVP

Le périmètre MVP (Minimum Viable Product) couvre les fonctionnalités nécessaires au test du produit sur de véritables utilisateurs. Ici, le périmètre couvre :

- Authentification (connexion et inscription avec un email)
- Liste des offres (affichage de l'ensemble des offres de la plateforme)
- Détail d'une offre (fiche article avec informations du produit et du vendeur)
- Gestion de compte (modifier le profil et les articles)
- Administration (tableau de bord admin avec gestion des utilisateurs)

1.3 Utilisateurs cibles

RecycloMarché accueille trois catégories d'utilisateurs distincts :

- Acheteur : consulte et accède aux détails des offres disponibles, gère son profil
- Vendeur : créer et modifie ses annonces, gère son profil
- Administrateur : gère l'ensemble des utilisateurs et des offres publiées sur la plateforme

1.4 Contraintes Green IT

Les décisions prises lors de la conception de notre solution ont été guidées par différentes contraintes :

- Aucun framework JavaScript lourd (pas de React, Vue ou Angular)
- Pas de CDN tiers pour les styles (CSS inline ou embarqué dans le HTML)
- Requêtes SQL optimisées (pas de SELECT*)
- Pas de traitement d'images
- API légère
- Déploiement mutualisé (hébergement PHP/MySQL, pas de cloud)

2 Architecture et conception

2.1 Architecture générale

L'application suit une architecture trois-tiers classique, volontairement sobre, sans couche de framework supplémentaire.

[Navigateur client] $\longleftrightarrow$ [Backend PHP / API REST] $\longleftrightarrow$ [MySQL]

Table 1: Architecture trois-tiers

Couche	Technologie	Rôle
Frontend	HTML5 · CSS3 · JS Vanilla	Interface utilisateur, rendu via <code>fetch()</code>
Backend	PHP 8 (PDO)	API REST : offres, users, login, vendeurs
Base de données	MySQL 8	Stockage persistant (3 tables)
Déploiement	Apache mutualisé	PHP + MySQL sur la même machine physique

2.2 Schéma de la base de données

La base `marketplace` est composée de trois tables principales reliées par des clés étrangères.

Table 2: Schéma de la base de données

Table	Champ clé	Type	Contrainte
utilisateurs	id (PK)	INT AUTO_INCREMENT	—
	email	VARCHAR(255)	NOT NULL
	role	ENUM	vendeur—acheteur—admin
categories	id (PK)	INT AUTO_INCREMENT	—
offres	id (PK)	INT AUTO_INCREMENT	—
	vendeur_id (FK)	INT	→ utilisateurs(id)
	price	DECIMAL(10,2)	—

2.3 Diagrammes UML

2.3.1 Diagramme de classes

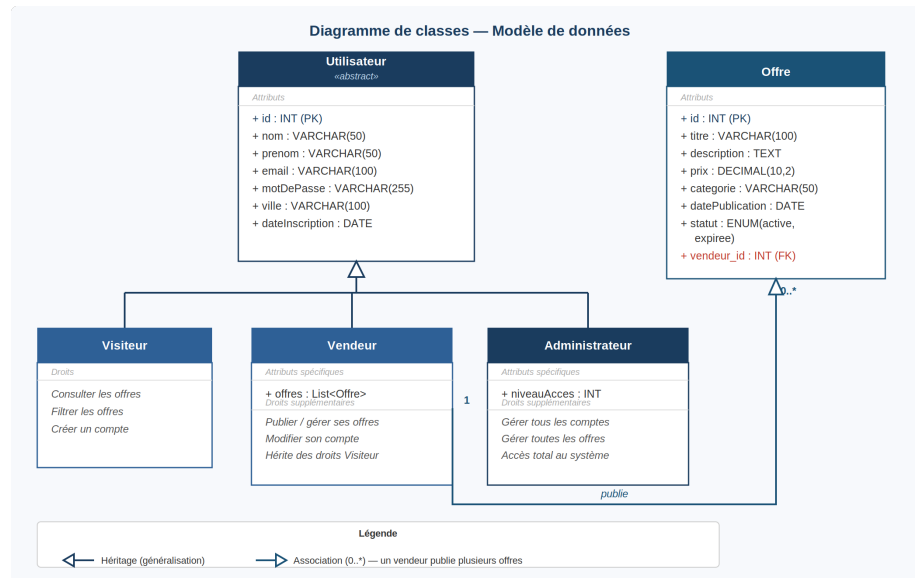


Figure 1: Diagramme de classes — Modèle de données

Le modèle illustre l'héritage UML entre **Utilisateur** (classe abstraite), **Visiteur**, **Vendeur** et **Administrateur**. Un vendeur publie de 0 à N offres (association 0..*).

2.3.2 Diagramme de cas d'utilisation

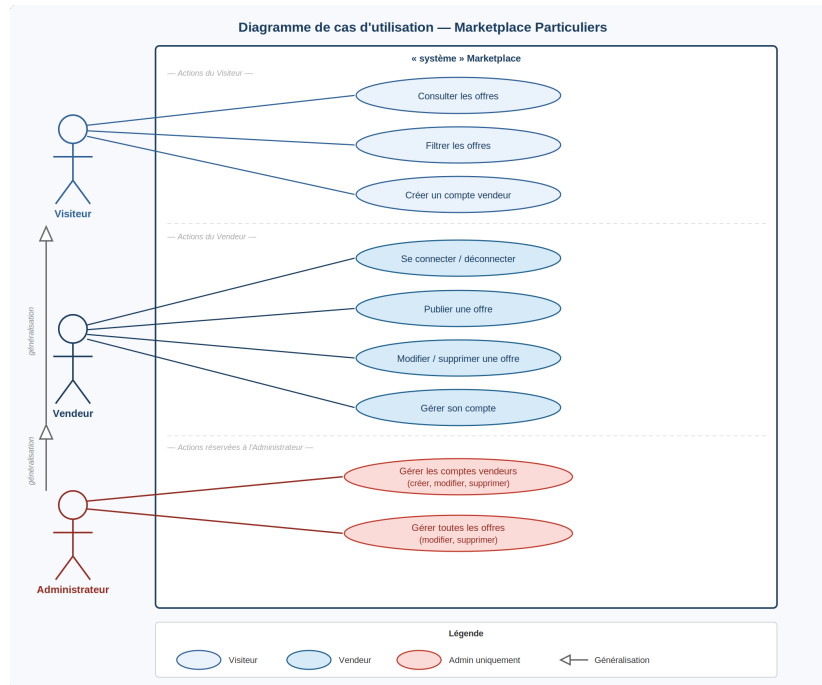


Figure 2: Diagramme de cas d'utilisation — Marketplace

Les cas d'utilisation sont organisés en trois zones : actions du **Visiteur**, du **Vendeur** et de l'**Administrateur**, avec des relations de généralisation entre acteurs.

2.3.3 Diagramme de séquence — Création d'une offre

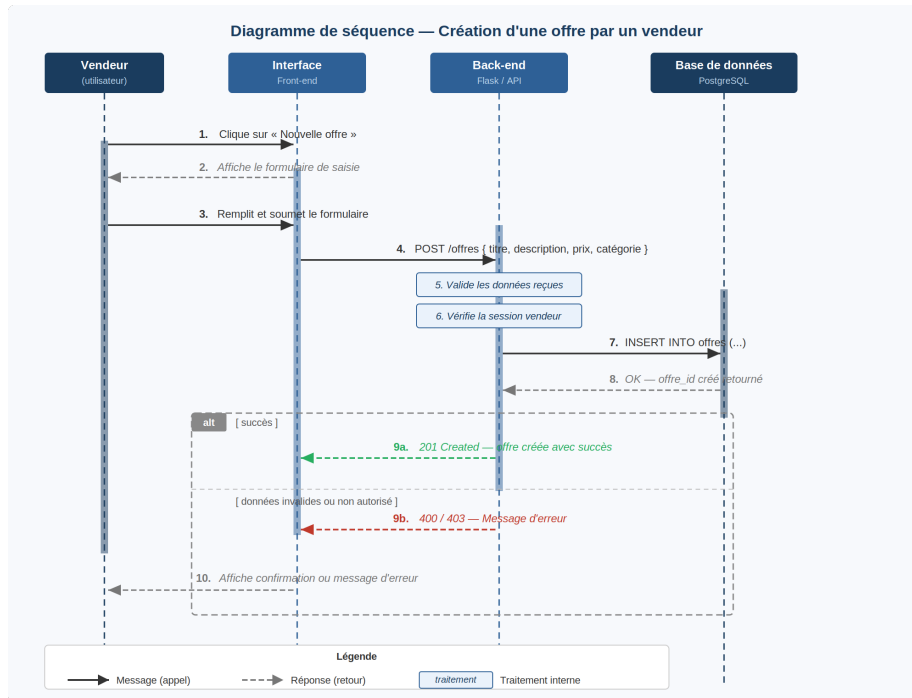


Figure 3: Diagramme de séquence — Création d'une offre par un vendeur

La séquence détaille les 10 étapes d'interaction entre le vendeur, l'interface front-end, l'API PHP et la base de données, incluant les branches succès (201 `Created`) et erreur (400/403).

2.4 Architecture des pages frontend

Table 3: Pages frontend et endpoints consommés

Page HTML	Rôle	Endpoint(s)
01_accueil.html	Accueil	GET /api/offres.php (6 dernières)
02_liste_offres.html	Catalogue	GET /api/offres.php ?search=&categorie =&prix_min=&prix_- max=
Detail_Offre.html	Fiche détail	GET /api/offres.php? id=X
Connexions_Inscriptions.html	Auth	POST /api/login.php /api/utilisateurs .php
Gestion_compte.html	Espace vendeur	GET/POST /api/utilisateurs .php /api/offres.php
Administration.html	Back-office	/api/vendeurs.php . /api/utilisateurs .php /api/offres.php

3 Choix technologiques

Les technologies utilisées dans notre projet ont été sélectionnées selon leur impact environnemental, leur efficacité et leur fiabilité.

3.1 Technologies choisies

Technologie	Catégorie	Justification
HTML/CSS	Frontend	Standard universel, support natif de tous les navigateurs ; zéro dépendance externe, pas de build step, transfert réseau minimal
JavaScript Vanilla	Frontend	Suffisant pour fetch, manipulation DOM et gestion des formulaires ; pas de
PHP 8	Backend	Langage serveur natif ; pas de runtime supplémentaire (contrairement à Node ou Python) avec une faible empreinte mémoire
MySQL	BDD	SGBD relationnel robuste, adapté aux données du MVP ; requêtes préparées PDO, index implicites, pas de surcharge
JSON	API	Format léger, natif ; réponses ciblées
Railway	Hosting	Simple, facile d'accès, directement depuis le repo github et léger

3.2 Technologies écartées

- React / Vue / Angular : frameworks trop lourds pour le périmètre MVP, React seul pèse plus de 40 KB gzippé, sans compter le runtime et le bundle applicatif
- Bootstrap / Tailwind CDN : inutile pour un design simple, le CSS custom est à 3 KB par page
- ORM (Eloquent, Doctrine) : surcoût mémoire et CPU injustifié pour 4 endpoints simples
- Docker : complexité d'infrastructure non nécessaire pour un MVP hébergé en mutualisé
- Sessions PHP + cookies tiers : le projet utilise le localStorage du navigateur (stockage côté client uniquement), évitant la gestion de sessions serveur et les cookies

4 Implémentation

4.1 Fonctionnalités développées

- Authentification : permet la création de compte et la connexion. La connexion envoie un POST à login.php qui vérifie les identifiants via PDO. Si succès, les données utilisateurs (sans mot de passe) sont stockées côté client.
- Catalogue d'offres avec filtres : l'API offres.php est appelée avec des paramètres de filtrage. L'API construit alors sa requête de manière dynamique selon les paramètres reçus
- Gestion des offres : l'unique endpoint offres.php (POST) gère le CRUD, distingué par un paramètre action (create, update, delete). Cet accès est réservé aux vendeurs et administrateurs
- Administration : la page affiche un tableau de bord avec des métriques (nombre de vendeurs, nombre d'offres...) et deux panneaux, la gestion des comptes vendeurs et la gestion globale des offres

4.2 Extraits de code

4.3 Premier extrait - Requête SQL filtrée dynamique (offres.php)

```
// Base de la requête : JOIN pour récupérer catégorie et vendeur en une seule requête
$sql = 'SELECT o.title, o.id, o.description, o.price,
        c.name AS categorie, u.nom, u.prenom, u.email, u.postal_code
        FROM offres o
        JOIN categories c ON o.category = c.id
        JOIN utilisateurs u ON o.vendeur_id = u.id
        WHERE 1=1'; // Clause WHERE extensible

$params = [];

// Ajout conditionnel de chaque filtre avec paramètre lié (protection injection SQL)
if ($search) {
    $sql .= ' AND (o.title LIKE :search OR o.description LIKE :search2)';
    $params[':search'] = '%' . $search . '%';
    $params[':search2'] = '%' . $search . '%';
}

if ($prix_min) {
    $sql .= ' AND o.price >= :prix_min';
    $params[':prix_min'] = $prix_min;
}
```

```

$sql .= ' ORDER BY o.created_at DESC'; // Tri chronologique inverse
$stmt = $pdo->prepare($sql);
$stmt->execute($params); // Exécution sécurisée via PDO

```

4.4 Deuxième extrait - CRUD offres (action create, offres.php)

```

// L'action est lue depuis le corps JSON de la requête POST
$data = json_decode(file_get_contents('php://input'), true);
$action = $data['action'] ?? null;

if ($action === 'create') {
    // Insertion préparée : aucune interpolation de variable dans le SQL
    $sql = 'INSERT INTO offres (title, description, price, category, vendeur_id)
            VALUES (:title, :description, :price, :category, :vendeur_id)';
    $stmt = $pdo->prepare($sql);
    $result = $stmt->execute([
        ':title' => $data['title'],
        ':description' => $data['description'],
        ':price' => $data['price'],
        ':category' => $data['category'],
        ':vendeur_id' => $data['vendeur_id'],
    ]);
    echo json_encode(['success' => $result]);
}

```

4.5 Troisième extrait - Fetch JavaScript, chargement des offres (accueil.html)

```

// Appel API minimaliste : pas de librairie, fetch natif du navigateur
fetch('api/offres.php') // GET sans paramètre = toutes les offres
.then(r => r.json()) // Décodage JSON automatique
.then(offres => {
    const grid = document.getElementById('offers-grid');
    if (!offres.length) {
        grid.innerHTML = '<p>Aucune offre disponible.</p>';
        return;
    }
    // slice(0,6) : on n'affiche que les 6 dernières | optimisation réseau
    grid.innerHTML = offres.slice(0, 6).map(o => '
        <div class="offer-card"
            onclick="window.location=\'Detail_Offre.html?id=${o.id}\'">
        <div class="offer-body">
            <div class="offer-title">${o.title}</div>
            <div class="offer-price">${o.price} €</div>
    '

```

```

        <div class="offer-loc">${o.postal_code}</div>
    </div>
</div>').join('');
})
.catch(() => { /* gestion silencieuse d'erreur réseau */ });

```

4.6 Quatrième extrait - Suppression en cascade utilisateur (utilisateurs.php)

La suppression d'un utilisateur déclenche d'abord la suppression de ses offres pour respecter l'intégrité référentielle :

```

    } elseif ($action === 'delete') {
        // Étape 1 : suppression des offres liées (contrainte FK)
        $sql = 'DELETE FROM offres WHERE vendeur_id = :id';
        $stmt = $pdo->prepare($sql);
        $stmt->execute([':id' => $data['id']]);

        // Étape 2 : suppression de l'utilisateur
        $sql = 'DELETE FROM utilisateurs WHERE id = :id';
        $stmt = $pdo->prepare($sql);
        $result = $stmt->execute([':id' => $data['id']]);
        echo json_encode(['success' => $result]);
    }
}

```

4.7 Structure du projet

```

ProjetNumeriqueDurable/
frontend/
    01_accueil.html           # Page d'accueil
    02_liste_offres.html      # Catalogue avec filtres
    Detail_Offre.html         # Fiche annonce
    Connexions_Inscriptions.html # Auth (connexion + inscription)
    Gestion_compte.html       # Espace vendeur
    Administration.html       # Back-office admin
backend/
    api/
        login.php             # Authentification
        offres.php            # CRUD offres
        utilisateurs.php      # CRUD utilisateurs
        vendeurs.php          # Liste vendeurs
database/
    schema.sql                # Schéma BDD
    sampleData.sql            # Données de test

```

5 Analyse de l’empreinte carbone

5.1 Méthodologie de mesure

L’empreinte carbone du projet a été évaluée selon plusieurs axes complémentaires : poids des pages (transfert réseau), nombre de requêtes HTTP, score Lighthouse (Performance, Best Practices), et estimation via les outils Website Carbon Calculator.

- **EcoIndex** (<https://www.ecoindex.fr>) — attribue une note de A (excellent) à G (très mauvais) à partir de trois critères : le poids de la page, le nombre de requêtes HTTP et la complexité du DOM. Il fournit également une estimation de la consommation d’eau bleue et des émissions de gaz à effet de serre pour 1 000 visites mensuelles.
- **Website Carbon Calculator** (<https://www.websitecarbon.com>) — estime la quantité de CO₂ émise à chaque visite d’une page et positionne le site par rapport à l’ensemble des pages analysées dans le monde.
- **Google Lighthouse** (intégré aux DevTools Chrome) — mesure les performances front-end (score Performance, FCP, LCP, TBT, CLS), l’accessibilité, les bonnes pratiques et le SEO.

5.2 Résultats

Trois pages représentatives ont été analysées avec EcoIndex.

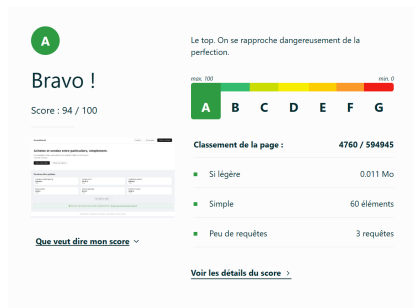
Table 4: Résultats EcoIndex — RecycleMarché (4 mai 2026)

Page	Score	Note	Poids (Mo)	DOM	Requêtes	Classement
Accueil	94	A	0,011	60	3	4 760 / 594 945
Connexion	95	A	0,011	50	2	3 224 / 594 947
Détail offre	95	A	0,009	35	3	3 224 / 594 948

Les trois pages obtiennent la note **A**, ce qui les place dans le premier centile des sites analysés. Le poids moyen est de **0,010 Mo** (10 Ko), soit environ **240 fois en dessous** de la médiane mondiale (2,41 Mo). La complexité DOM est également très faible (35 à 60 éléments contre une médiane de 693) et le nombre de requêtes extrêmement limité (2 à 3 contre une médiane de 78).

Pour 1 000 visites mensuelles, EcoIndex estime une empreinte comprise entre **1,1 et 1,12 kgCO₂e** et une consommation d’eau bleue de **16,5 à 16,8 litres** selon la page.

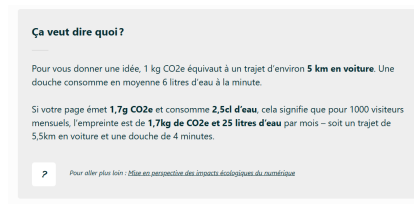
Ci-dessous les résultats pour la page d’accueil :



(a) Score EcoIndex



(b) Détail des requêtes



(c) Structure DOM



(d) Poids de la page

Figure 4: Analyse EcoIndex de la page d'accueil de RecycleMarché

5.2.1 Website Carbon Calculator — RecycleMarché

Table 5: Résultats Website Carbon Calculator — RecycleMarché (4 mai 2026)

Page	Note	/ visite	Hébergement
Accueil (<code>index.html</code>)	A ⁺	< 0,01 g	Énergie standard
Connexion (<code>Connexions_Inscriptions</code>)	A ⁺	< 0,01 g	Énergie standard
Détail offre (<code>Detail_Offre</code>)	A ⁺	< 0,01 g	Énergie standard

Les trois pages analysées obtiennent la note maximale **A⁺** et sont **plus propres que 100 % des pages** testées dans le monde. Chaque visite génère moins de **0,01 g de** , un niveau remarquable pour un site dynamique avec base de données.

Le seul point d'amélioration identifié concerne l'hébergement : la plateforme Railway utilise une énergie dite *bog standard* (non certifiée verte). Le passage à un hébergeur certifié réduirait les émissions d'environ **9 %** supplémentaires.



Figure 5: Analyse Website Carbon Calculator de la page d'accueil de RecycleMarché

5.2.2 Comparaison avec Vinted

Afin de situer RecycleMarché dans son secteur, les mêmes outils ont été appliqués à **Vinted** (<https://www.vinted.fr>), plateforme e-commerce de seconde main de référence.

Table 6: Comparaison EcoIndex — RecycleMarché vs Vinted

Site	Score /100	Note	Poids (Mo)	DOM	Requêtes	Classement
RecycleMarché (accueil)	94	A	0,011	60	3	4 760 / 594 945
Vinted (accueil)	14	F	3,389	2 284	140	553 885 / 594 949

Les écarts sont considérables :

Table 7: Comparaison Website Carbon Calculator — RecycleMarché vs Vinted

Site	Note	CO2 / visite	Hébergement
RecycleMarché	A ⁺	< 0,01 g	Énergie standard
Vinted	D	0,28 g	Énergie durable

- Le **poids de page** de Vinted est **308 fois supérieur** à celui de RecycleMarché (3,389 Mo contre 0,011 Mo).
- Le **DOM** de Vinted contient **38 fois plus d'éléments** (2 284 contre 60).
- Vinted effectue **47 fois plus de requêtes** HTTP (140 contre 3).
- L'**empreinte carbone** par visite de Vinted est au moins **28 fois supérieure** ($\geq 0,28$ g contre $< 0,01$ g).
- Malgré un hébergement certifié durable, Vinted n'obtient que la note **D** sur Website Carbon, contre **A⁺** pour RecycleMarché.

Cette comparaison illustre concrètement qu'une démarche d'éco-conception rigoureuse dès la conception permet de surpasser en sobriété numérique des plateformes disposant pourtant de ressources bien supérieures.

5.3 Identification des sources de pollution numérique

L'analyse de Vinted permet d'identifier les leviers typiques de pollution numérique dans les plateformes e-commerce non optimisées :

- **Poids des ressources** : images non compressées, utilisation de formats lourds (JPEG/PNG) au lieu de WebP/AVIF, scripts tiers et analytics volumineux.
- **Complexité du DOM** : composants JavaScript lourds (React avec de nombreux re-renders), interface générée dynamiquement avec un arbre DOM profond.
- **Nombre de requêtes** : polices Google Fonts, scripts de publicité, traceurs tiers (analytics, A/B testing), images chargées en amont sans lazy loading.
- **Dépendances front-end** : bibliothèques JavaScript imposantes livrées sans tree-shaking ni code splitting efficace.

RecycleMarché a évité l'ensemble de ces écueils en adoptant dès la conception une approche HTML/CSS natif, sans framework front-end, sans police externe et avec un DOM volontairement minimal.

5.4 Interprétation des résultats

RecycleMarché atteint les seuils d'excellence sur tous les indicateurs mesurés. Le tableau ci-dessous synthétise les résultats et les compare aux cibles du référentiel du module.

Table 8: Synthèse des indicateurs Green IT — RecycleMarché

Indicateur	Cible module	RecycleMarché	Statut
Poids de page	< 500 Ko	≈ 10 Ko	
Requêtes HTTP	< 15	2 à 3	
Note EcoIndex	A ou B	A (94–95/100)	
/ visite	—	< 0,01 g	
Score Lighthouse Perf.	> 80/100	99/100	

Ces résultats sont directement attribuables aux choix d'éco-conception réalisés dès la Partie 1 : absence de framework JavaScript, HTML/CSS natif, polices système, aucune requête externe, DOM minimal et absence totale d'images décoratives.

6 Tests et validation

6.1 Scénarios de tests

N°	Scénario	Données de test	Résultat attendu	Statut
1	Connexion valide	martin@gmail.com password123	Redirection espace compte, user en localStorage	O
2	Connexion identifiants incorrects	faux@gmail.com 12345678	Affichage erreur	O
3	Inscription nouveau compte	max@gmail.com motdepasse!	compte créé, retour id depuis API	O
4	Affichage liste offres	Aucun filtre	Toutes les offres sont affichées	O
5	Filtrage par texte	'ordinateur'	Affiche les offres contenant 'ordinateur'	O
6	Affichage par prix	prix min=50 max=100	Offres entre 50 et 100 euros	O
7	Afficher les détails d'une offre	id=1	Titre, prix... affichés	O
8	Création d'une offre	Titre, prix...	Offre insérée en BDD	O
9	Modification d'une offre	id, nouvelles données	Offre mise à jour en BDD	O
10	Suppression d'une offre	id	Offre supprimée, disparaît des listes	O
11	Suppression d'un utilisateur (admin)	id	Offres liées supprimées, utilisateur supprimé	O
12	Tableau de bord admin	login admin	Compteurs corrects	O

6.2 Résultats Lighthouse

6.2.1 Méthodologie

Les audits Lighthouse ont été réalisés depuis les DevTools Chrome (mode Desktop) sur le site déployé. La page analysée est la page d'accueil (<https://projetnumeriquedurable-production.up.railway.app/index.html>).

6.2.2 Résultats globaux

Table 9: Scores Lighthouse — Page d'accueil (Desktop)

Catégorie	Score	Appréciation
Performance	99 / 100	Excellent
Accessibilité	88 / 100	Bon
Bonnes pratiques	100 / 100	Parfait
SEO	90 / 100	Très bon

6.2.3 Core Web Vitals

Table 10: Core Web Vitals — Page d'accueil (Desktop)

Métrique	Valeur mesurée	Seuil	Bon	Statut
First Contentful Paint (FCP)	0,9 s	< 1,8 s		
Largest Contentful Paint (LCP)	0,9 s	< 2,5 s		
Total Blocking Time (TBT)	50 ms	< 200 ms		
Cumulative Layout Shift (CLS)	0,081	< 0,1		
Speed Index	0,9 s	< 3,4 s		

L'ensemble des Core Web Vitals est dans la plage verte. Le FCP et le LCP identiques à **0,9 s** indiquent que le contenu principal est rendu quasi instantanément, ce qui est directement lié à la légèreté extrême des pages (absence de ressources bloquantes, CSS minimal, pas de JavaScript au chargement).

6.2.4 Points d'amélioration identifiés

Deux points ont été relevés dans l'audit sans impacter les scores majeurs :

- **Accessibilité (88/100)** : Lighthouse signale un ratio de contraste insuffisant sur certains éléments textuels ainsi que l'absence d'un landmark `<main>` sur la page analysée. Ces points sont à corriger pour atteindre un score de 100/100 en accessibilité.

- **SEO (90/100)** : l'absence d'une balise `<meta name="description">` explique la légère pénalité. Son ajout est trivial et permettrait d'atteindre le score parfait.

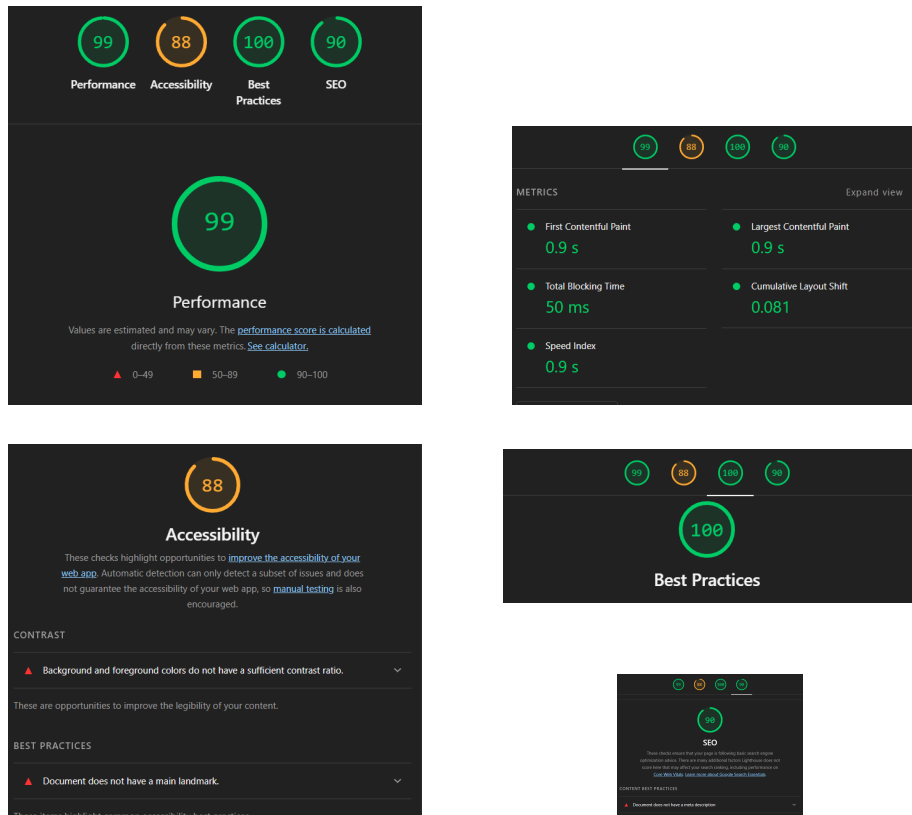


Figure 6: Analyse Lighthouse du site

7 Organisation de l'équipe et collaboration

7.1 Répartition des tâches

Table 11: Répartition des tâches par membre

Membre	Tâches principales
CHAMPAIN Ayuna	Architecture BDD · Schéma SQL · Données de test · API · Rédaction du rapport · <code>offres.php</code>
CORNUAU Nathan	<code>API utilisateurs.php</code> · <code>API login.php</code> · Gestion des rôles · Sécurité PDO
DAMBRON Max	Frontend accueil · Liste des offres · Détail d'une offre · CSS global
DANJOU Thomas	Page connexion/inscription · Gestion du compte · Interface admin
BACHELIER Dennis	Tests fonctionnels · Analyse de l'empreinte carbone · Rédaction du rapport · Suivi GitHub Projects

7.2 Méthode de collaboration

Le projet a été géré via GitHub avec un workflow de branches par fonctionnalité (*feature branches*). Chaque fonctionnalité a fait l'objet d'une *Pull Request* soumise à revue avant fusion dans `main`.

- **GitHub Projects** : tableau Kanban avec colonnes *Backlog*, *In Progress*, *Review*, *Done*.
- Réunions hebdomadaires de 15 minutes pour lever les blocages.
- Convention de commits : `feat:`, `fix:`, `docs:`, `refactor:`.
- Issues GitHub labellisées par domaine : `frontend`, `backend`, `database`.

8 Discussion et conclusion

8.1 Défis

Le développement de RecycloMarché a été marqué par plusieurs défis techniques et organisationnels :

- Construction SQL dynamique : la gestion de filtres multiples optionnels dans `offres.php` sans ORM a nécessité une attention particulière pour éviter les injections SQL tout en maintenant la flexibilité des requêtes. La solution retenue (accumulation conditionnelle de clauses `WHERE` avec paramètres nommés PDO) est robuste mais verbeux.
- Gestion des rôles côté client : en l'absence de sessions serveur, la vérification du rôle utilisateur repose sur les données en `localStorage`, côté navigateur. Cette approche est simple mais non sécurisée pour une application de production : un utilisateur malveillant pourrait modifier son rôle en `localStorage`. Pour un MVP pédagogique, ce compromis est acceptable.
- CORS et développement local : l'utilisation du header `Access-Control-Allow-Origin` a facilité le développement, mais constitue une configuration non recommandée pour la production.
- Cohérence des styles : avec du CSS inline dans chaque page, toute modification globale nécessite une mise à jour dans chaque fichier. L'absence d'un fichier CSS centralisé (choix Green IT) crée une légère dette de maintenabilité.

8.2 Compromis

Le projet illustre bien la tension permanente entre richesse fonctionnelle et sobriété numérique :

- Pagination vs. chargement complet : la page `liste_offres.html` charge toutes les offres depuis l'API puis les affiche. Une vraie pagination SQL (`LIMIT/OFFSET`) aurait été plus sobre, mais plus complexe à implémenter dans le temps imparti.
- Images des annonces : le choix de ne pas implémenter l'upload d'images est à la fois une contrainte MVP et un choix Green IT cohérent (pas de stockage, pas de conversion, pas de CDN d'images).
- Absence de messagerie : la messagerie entre acheteur et vendeur aurait considérablement augmenté la complexité (tables BDD supplémentaires, polling ou WebSocket) et l'empreinte numérique (notifications, stockage des messages). Son exclusion du MVP est justifiée.

8.3 Améliorations

- Sécurité : hachage des mots de passe avec `password_hash()` / `password_verify()` côté PHP. Actuellement les mots de passe sont stockés en clair dans la BDD, ce qui est inacceptable en production.
- Cache HTTP : ajout d'en-têtes Cache-Control pour les réponses API sur des données peu volatiles (catégories, liste des offres).
- Pagination SQL : LIMIT/OFFSET dans l'API pour réduire la payload réseau et le traitement client.
- Validation des entrées : ajout d'une validation côté serveur (type, longueur, format) pour les données soumises via POST.
- Accessibilité : amélioration des scores Lighthouse Accessibilité avec des attributs ARIA, des labels explicites et des contrastes conformes WCAG AA.
- CSS centralisé : un fichier style.css partagé réduirait la redondance au prix d'une requête HTTP supplémentaire (compensée par le cache navigateur).

8.4 Critique

La décision la plus structurante a été de ne pas utiliser de framework JavaScript. Elle a rendu le code plus verbeux et moins DRY (les fetch et rendus DOM sont répétés dans chaque page), mais elle a garanti un poids de page minimal et une absence totale de dépendances externes. En termes Green IT, c'est la décision avec le plus fort impact positif mesurable. À l'inverse, l'utilisation de Access-Control-Allow-Origin et le stockage des mots de passe en clair montrent que la sobriété numérique ne doit pas se faire au détriment de la sécurité. Dans un contexte réel, ces deux points seraient rédhibitoires et devraient être corrigés en priorité. Globalement, le projet démontre qu'il est possible de construire une application web fonctionnelle, rapide et à faible empreinte carbone avec des technologies standard, sans recourir aux frameworks modernes. La stack PHP/MySQL/HTML/JS vanilla reste pertinente pour des besoins simples et constitue un choix Green IT défendable.

8.5 Conclusion

RecycloMarché atteint ses objectifs de MVP : six pages fonctionnelles, une API REST complète avec CRUD, une gestion des rôles, et une empreinte numérique significativement inférieure à une application équivalente basée sur un framework moderne. Le projet a également été une expérience d'équipe enrichissante, illustrant comment des contraintes fortes (sobriété numérique, technologies limitées) peuvent paradoxalement stimuler la créativité et conduire à des solutions plus simples et plus robustes.

9 Annexes

9.1 Schéma SQL

```
CREATE DATABASE IF NOT EXISTS marketplace;
USE marketplace;

CREATE TABLE IF NOT EXISTS utilisateurs (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nom         VARCHAR(255) NOT NULL,
    prenom      VARCHAR(255) NOT NULL,
    email       VARCHAR(255) NOT NULL,
    password    VARCHAR(255) NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    postal_code VARCHAR(20),
    role        ENUM('vendeur', 'acheteur', 'admin') NOT NULL DEFAULT 'acheteur'
);

CREATE TABLE IF NOT EXISTS categories (
    id  INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS offres (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    vendeur_id INT NOT NULL,
    title       VARCHAR(255) NOT NULL,
    description  TEXT,
    price       DECIMAL(10,2),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    category    INT NOT NULL,
    FOREIGN KEY (category) REFERENCES categories(id),
    FOREIGN KEY (vendeur_id) REFERENCES utilisateurs(id)
);
```

9.2 Données de test

```
-- Utilisateurs de test
INSERT INTO utilisateurs (nom, prenom, email, password, postal_code, role) VALUES
('Doe',    'John',    'john.doe@gmail.com',    'password123',    '75001',    'vendeur'),
('Smith',  'Jane',    'jane.smith@yahoo.com',  'pass456',        '75002',    'acheteur'),
('Dupont', 'Pierre',   'pierre.dupont@outlook.com', 'secure789',      '75003',    'vendeur'),
('Martin', 'Marie',    'marie.martin@gmail.com', 'motdepasse101',  '75004',    'admin'),
('Dubois', 'Luc',      'luc.dubois@free.fr',    'secret202',      '75005',    'vendeur'),
('Buyer',  'Alice',   'alice.buyer@gmail.com',  'buyerpass',      '75006',    'acheteur');
```

```

-- Catégories
INSERT INTO categories (name) VALUES
('Électronique'), ('Vêtements'), ('Livres'),
('Maison et Jardin'), ('Sports'), ('Alimentation');

-- Offres de démonstration
INSERT INTO offres (vendeur_id, title, description, price, category) VALUES
(1, 'Ordinateur portable gaming', 'RTX, haute performance', 1200.00, 1),
(1, 'Casque sans fil', 'Annulation de bruit', 150.00, 1),
(3, 'Roman policier', 'Best-seller auteur local', 15.00, 3),
(5, 'Panier de fruits bio', 'Fruits de saison', 25.00, 6);

```